

Programación III — Solucionario Oficial

Resolución Guía Práctica: Ejercicios_Programacion_III_Algoritmos_2.pdf

Cátedra: Programación III

Tema: Eficiencia, Conteo y Notación Asintótica

Sección 1: Conteo de Operaciones y Análisis de Código

EJERCICIO 1: Cálculo de Operaciones Elementales y Peor Caso

```
public boolean contieneDuplicados(int[] v) {  
    int n = v.length;           // LÍNEA 2  
    for (int i = 0; i < n; i++)   // LÍNEA 3  
        for (int j = i + 1; j < n; j++) { // LÍNEA 4  
            if (v[i] == v[j]) {    // LÍNEA 5  
                return true;      // LÍNEA 6  
            }  
        }  
    return false;               // LÍNEA 9  
}
```

✓ RESOLUCIÓN PASO A PASO:

1. Instancia de Peor Caso:

El peor caso ocurre cuando el algoritmo recorre la totalidad de las iteraciones anidadas sin retornar de forma anticipada. Esto sucede cuando el arreglo **no contiene elementos duplicados** (o cuando los duplicados están al final del arreglo). En este escenario, el método siempre llega a la Línea 9 y retorna false.

2. Cálculo Exacto de Operaciones Elementales $T(n)$:

Evaluamos el número de iteraciones de los bucles anidados:

- Para $i = 0$, el bucle j se ejecuta $n - 1$ veces.
- Para $i = 1$, el bucle j se ejecuta $n - 2$ veces.
- ...
- Para $i = n - 2$, el bucle j se ejecuta 1 vez.

Sumando el total de iteraciones del bucle interno:

$$N_{\text{iteraciones}} = \sum_{i=0}^{n-2} (n - 1 - i) = (n - 1) + (n - 2) + \dots + 1 = [n(n - 1)] / 2 = (n^2 - n) / 2$$

Contando asignaciones, incrementos, comparaciones e instrucciones elementales:

$$T(n) = c_1 \cdot n^2 + c_2 \cdot n + c_3$$

3. Notación Asintótica Big-O:

Conservando únicamente el término dominante de mayor orden (n^2) y eliminando constantes y coeficientes multiplicativos:

Complejidad Temporal en Peor Caso: $O(n^2)$ (Cuadrática)

EJERCICIO 2: El Dilema del Condicional y la Regla del Máximo

```
public void procesarMatriz(int[][] matriz, boolean modoRapido) {
    int n = matriz.length;
    if (modoRapido) {
        // RAMA A: Recorrido de la diagonal principal
        for (int i = 0; i < n; i++) {
            System.out.println(matriz[i][i]);
        }
    } else {
        // RAMA B: Recorrido completo de la matriz
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                System.out.println(matriz[i][j]);
            }
        }
    }
}
```

✓ RESOLUCIÓN PASO A PASO:

1. Análisis de la Regla del Máximo para Condicionales:

La regla establece que $T_{if}(n) = T_{condición} + \max(T_{then}(n), T_{else}(n))$.

- **Rama A (modoRapido == true):** Bucle único de 0 a $n - 1$. Costo: $O(n)$.
- **Rama B (modoRapido == false):** Bucles anidados de tamaño $n \times n$. Costo: $O(n^2)$.

2. Complejidad Temporal Total en Peor Caso:

Aplicando la regla del máximo:

$$T(n) = O(1) + \max(O(n), O(n^2)) = O(n^2)$$

Conclusión: El peor caso depende exclusivamente de la ejecución de la ****Rama B**** (cuando `modoRapido == false`), que impone un orden de crecimiento cuadrático $O(n^2)$.

Sección 2: Experimentación y Medición Práctica

EJERCICIO 3: Medición Empírica vs. Medición Teórica

Implementación de los algoritmos de Búsqueda Lineal ($O(n)$) y Comparación Par a Par ($O(n^2)$) en Java.

✓ IMPLEMENTACIÓN JAVA Y RESULTADOS:

```
public class MedicionMaximo {
    // Algoritmo A: Búsqueda Lineal - O(n)
    public static int maximolineal(int[] v) {
        int max = v[0];
        for (int i = 1; i < v.length; i++) {
            if (v[i] > max) max = v[i];
        }
        return max;
    }

    // Algoritmo B: Comparación Par a Par - O(n²)
    public static int maximoCuadratico(int[] v) {
        int n = v.length;
        for (int i = 0; i < n; i++) {
            boolean esMax = true;
            for (int j = 0; j < n; j++) {
                if (v[j] > v[i]) {
                    esMax = false;
                    break;
                }
            }
            if (esMax) return v[i];
        }
        return v[0];
    }
}
```

Tabla Experimental Típica de Tiempos Ejecutados:

Tamaño de Entrada (n)	Tiempo Algoritmo A — O(n)	Tiempo Algoritmo B — O(n²)
n = 1.000	~ 0.05 ms	~ 2.5 ms
n = 10.000	~ 0.40 ms	~ 220 ms
n = 100.000	~ 3.50 ms	~ 22.000 ms (~22 seg)

Análisis y Conclusión:

Al multiplicar el tamaño de la entrada por 10 (de 1.000 a 10.000 o de 10.000 a 100.000), el tiempo de ejecución del Algoritmo B se **multiplica aproximadamente por 100 (10^2)**. Esto confirma la teoría asintótica: al ser un algoritmo de complejidad $O(n^2)$, multiplicar la entrada por un factor k incrementa el tiempo por k^2 .

Sección 3: Notación Asintótica y Órdenes de Crecimiento

EJERCICIO 4: Clasificación y Ordenamiento de Funciones

- Algoritmo A: $T_A(n) = 100n^2 + 50n + 12$
- Algoritmo B: $T_B(n) = 2^n + 10$
- Algoritmo C: $T_C(n) = 3n \log_2(n) + 1000$
- Algoritmo D: $T_D(n) = 50000$
- Algoritmo E: $T_E(n) = 2n + 5$

✓ RESOLUCIÓN PASO A PASO:

1. Complejidades Big-O Simplificadas:

- **Algoritmo A:** Dominante $100n^2 \Rightarrow O(n^2)$ (Cuadrática).
- **Algoritmo B:** Dominante $2^n \Rightarrow O(2^n)$ (Exponencial).
- **Algoritmo C:** Dominante $3n \log_2(n) \Rightarrow O(n \log n)$ (Lineárítmica).
- **Algoritmo D:** Término constante $50000 \Rightarrow O(1)$ (Constante).
- **Algoritmo E:** Dominante $2n \Rightarrow O(n)$ (Lineal).

2. Ordenamiento de Menor a Mayor Costo Temporal ($n \rightarrow \infty$):

Ordenando las clases de complejidad de menor a mayor crecimiento asintótico:

$$O(1) < O(n) < O(n \log n) < O(n^2) < O(2^n)$$

Secuencia Final de Algoritmos (del más eficiente al menos eficiente):

$$\text{Algoritmo D} < \text{Algoritmo E} < \text{Algoritmo C} < \text{Algoritmo A} < \text{Algoritmo B}$$